

Math

Josephus Queries

```
#include

using namespace std;
typedef pair pii;

int Q;
map f;

int solve(int N, int K){
    if(f[{N, K}]) return f[{N, K}];

    if(N == 1) return f[{N, K}] = 1;
    if(2*K <= N) return f[{N, K}] = 2*K;

    int newN = (N/2)+(N&1);
    int newK = K-N/2;

    if(N&1){
        int ans = solve(newN, newK);
        return f[{N, K}] = ((2*ans-3)+(N+1)) % (N+1);
    } else {
        return f[{N, K}] = 2*solve(newN, newK)-1;
    }
}

int main(){
    scanf("%d", &Q);
    for(int q = 0, N, K; q < Q; q++){
        scanf("%d %d", &N, &K);
        printf("%d\n", solve(N, K));
    }
}
```

Exponentiation

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

int N;
ll A, B;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
    }
}
```

```

        b >= 1;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%lld %lld", &A, &B);
        printf("%lld\n", fastpow(A, B));
    }
}

```

Exponentiation II

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

int N;
ll A, B, C;

ll fastpow(ll a, ll b, ll mod){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % mod;
        a = (a * a) % mod;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%lld %lld %lld", &A, &B, &C);
        printf("%lld\n", fastpow(A, fastpow(B, C, MOD-1), MOD));
    }
}

```

Counting Divisors

```

#include

using namespace std;
const int maxX = 1e6;

int N, x, d[maxX+1];

```

```

void init(){
    for(int i = 1; i <= maxX; i++)
        for(int j = i; j <= maxX; j += i)
            d[j]++;
}

int main(){
    scanf("%d", &N);

    init();
    for(int i = 0; i < N; i++){
        scanf("%d", &x);
        printf("%d\n", d[x]);
    }
}

```

Common Divisors

```

#include

using namespace std;
const int maxX = 1e6;

int N, a, cnt, d[maxX+1];

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &a);
        d[a]++;
    }
    for(int i = maxX; i >= 1; i--){
        cnt = 0;
        for(int j = i; j <= maxX; j += i)
            cnt += d[j];
        if(cnt >= 2){
            printf("%d\n", i);
            return 0;
        }
    }
}

```

Sum of Divisors

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, sum;

```

```

11 sigma(11 x){
    return ((x%MOD)*((x+1)%MOD)/2)%MOD;
}

int main(){
    scanf("%lld", &N);
    for(11 l = 1; l <= N; l++){
        11 r = N/(N/l);
        sum += (N/l)%MOD*(sigma(r)-sigma(l-1)+MOD)%MOD;
        sum = (sum+MOD)%MOD;
        l = r;
    }
    printf("%lld\n", sum);
}

```

Divisor Analysis

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e5;
const ll MOD = 1e9+7;

int N;
ll x[maxN], k[maxN];
ll tau, sigma, pi, mu;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++)
        scanf("%lld %lld", &x[i], &k[i]);

    tau = 1;
    for(int i = 0; i < N; i++)
        tau = (tau * (k[i]+1)) % MOD;

    sigma = 1;
    for(int i = 0; i < N; i++){
        ll numerator = (fastpow(x[i], k[i]+1)-1+MOD) % MOD;
        ll denominator = fastpow(x[i]-1, MOD-2);
        ll geoSum = numerator * denominator % MOD;
        sigma = (sigma * geoSum) % MOD;
    }

    pi = 1;
    mu = 1;
    for(int i = 0; i < N; i++){
        ll p = fastpow(x[i], k[i]*(k[i]+1)/2);
        mu = fastpow(mu, k[i]+1) * fastpow(p, pi) % MOD;
    }
}

```

```

        pi = (pi * (k[i]+1)) % (MOD-1);
    }

    printf("%lld %lld %lld\n", tau, sigma, mu);
}

```

Prime Multiples

```

#include

using namespace std;
typedef long long ll;
const double EPS = 0.001;
const int maxK = 20;

int K;
ll N, cnt, a[maxK];

int main(){
    scanf("%lld %d", &N, &K);
    for(int i = 0; i < K; i++)
        scanf("%lld", &a[i]);

    cnt = N;
    double RHS = log(N) + EPS;
    for(int mask = 0; mask < (1<

```

Counting Coprime Pairs

```

#include

using namespace std;
typedef long long ll;
const int maxX = 1e6+1;

ll ans;
int N, dp[maxX];
bool b[maxX];
vector primes;

void init(){
    fill(b+2, b+maxX, true);
    for(int i = 2; i*i < maxX; i++)
        if(b[i])
            for(int j = i*i; j < maxX; j += i)
                b[j] = false;
    for(int i = 2; i < maxX; i++)
        if(b[i])
            primes.push_back(i);
}

void compute(int x){

```

```

vector pf;
for(int p : primes){
    if(x == 1) break;
    else if(b[x]){
        pf.push_back(x);
        break;
    }

    if(x % p) continue;
    pf.push_back(p);
    while(x % p == 0)
        x /= p;
}

int K = (int) pf.size();
for(int mask = 0; mask < (1<

```

Next Prime

```

#include

using namespace std;
typedef long long ll;
const int sqrtN = 1e6+5;

bool prime[sqrtN];
vector p;

void init_primes(){
    for(int i = 2; i < sqrtN; i++) prime[i] = true;

    for(int i = 2; i * i <= sqrtN; i++)
        if(prime[i])
            for(int j = 2*i; j < sqrtN; j += i)
                prime[j] = false;

    for(int i = 2; i < sqrtN; i++)
        if(prime[i])
            p.push_back(i);
}

bool is_prime(ll n){
    for(int i = 0; i < (int) p.size() && p[i] < n; i++)
        if(n % p[i] == 0)
            return false;
    return true;
}

int main(){
    init_primes();

    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        ll n;
        scanf("%lld", &n);

        n++;
        while(!is_prime(n))
            n++;
    }
}

```

```

        printf("%lld\n", n);
    }
}

```

Binomial Coefficients

```

#include

using namespace std;
typedef long long ll;
const int maxA = 1e6;
const ll MOD = 1e9+7;

int N, a, b;
ll fact[maxA+1], inv[maxA+1];

ll inverse(ll x){
    ll res = 1;
    ll expo = MOD-2;
    while(expo > 0){
        if(expo&1)
            res = (res * x) % MOD;
        x = (x * x) % MOD;
        expo >>= 1;
    }
    return res;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i <= maxA; i++){
        fact[i] = i * fact[i-1] % MOD;
        inv[i] = inverse(fact[i]);
    }
}

int main(){
    init();
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d %d", &a, &b);
        printf("%lld\n", fact[a] * inv[b] % MOD * inv[a-b] % MOD);
    }
}

```

Creating Strings II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e6+5;

```

```

const ll MOD = 1e9+7;

int N, freq[26];
char S[maxN];
ll fact[maxN], inv[maxN];

ll inverse(ll x){
    ll res = 1;
    ll expo = MOD-2;
    while(expo > 0){
        if(expo&1)
            res = (res * x) % MOD;
        x = (x * x) % MOD;
        expo >>= 1;
    }
    return res;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fact[i] = i * fact[i-1] % MOD;
        inv[i] = inverse(fact[i]);
    }
}

int main(){
    scanf("%s", S);
    N = (int) strlen(S);
    init();

    for(int i = 0; i < N; i++)
        freq[(int) (S[i]-'a')]++;

    ll ans = fact[N];
    for(int i = 0; i < 26; i++)
        ans = ans * inv[freq[i]] % MOD;
    printf("%lld\n", ans);
}

```

Distributing Apples

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e6;
const ll MOD = 1e9+7;

int N, M;
ll fact[maxN], inv[maxN];

ll inverse(ll x){
    ll res = 1;
    ll expo = MOD-2;
    while(expo > 0){
        if(expo&1)
            res = (res * x) % MOD;
        x = (x * x) % MOD;
        expo >>= 1;
    }
}

```

```

    return res;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i < maxN; i++){
        fact[i] = i * fact[i-1] % MOD;
        inv[i] = inverse(fact[i]);
    }
}

ll choose(int n, int k){
    return fact[n] * inv[k] % MOD * inv[n-k] % MOD;
}

int main(){
    init();
    scanf("%d %d", &N, &M);
    printf("%lld\n", choose(N+M-1, M));
}

```

Christmas Party

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e6;
const ll MOD = 1e9+7;

int N;
ll dp[maxN+1];

void init(){
    dp[2] = 1;
    for(int i = 3; i <= maxN; i++)
        dp[i] = (i-1) * (dp[i-1] + dp[i-2]) % MOD;
}

int main(){
    init();
    scanf("%d", &N);
    printf("%lld\n", dp[N]);
}

```

Permutation Order

```

#include

using namespace std;
typedef long long ll;
const int maxN = 21;

```

```

ll fac[maxN];

void init(){
    fac[0] = 1;
    for(int i = 1; i < maxN; i++)
        fac[i] = fac[i-1] * i;
}

vector query_one(const vector& a, ll k){
    if(a.empty())    return {};

    int n = (int) a.size();
    int first_digit = (int) (k / fac[n-1]);

    ll new_k = k - (first_digit * fac[n-1]);
    vector new_a;
    for(int i = 0; i < n; i++)
        if(i != first_digit)
            new_a.push_back(a[i]);

    vector p = query_one(new_a, new_k);
    p.push_back(a[first_digit]);
    return p;
}

ll query_two(int n, const vector& p){
    ll k = 1;
    for(int i = 0; i < n; i++){
        ll place_value = fac[n-i-1];

        int num_smaller = 0;
        for(int j = 0; j < i; j++)
            if(p[j] < p[i])
                num_smaller++;

        int digit_in_place = p[i] - num_smaller - 1;

        k += place_value * digit_in_place;
    }
    return k;
}

int main(){
    init();

    int T;
    scanf("%d", &T);
    for(int t = 0, type, n; t < T; t++){
        scanf("%d %d", &type, &n);
        if(type == 1){
            ll k;
            scanf("%lld", &k);

            vector a;
            for(int i = 1; i <= n; i++)
                a.push_back(i);

            vector p = query_one(a, k-1);
            reverse(p.begin(), p.end());

            for(int i = 0; i < n; i++)
                printf("%d%c", p[i], (" \n")[i==n-1]);
        } else {
            vector p(n);
            for(int i = 0; i < n; i++)
                scanf("%d", &p[i]);

            printf("%lld\n", query_two(n, p));
        }
    }
}

```

```

    }
}

```

Permutation Rounds

```

#include

using namespace std;
typedef long long ll;
const int maxN = 2e5+1;
const ll MOD = (ll) 1e9+7;

int N, perm[maxN];
bool vis[maxN], is_prime[maxN];
vector primes;
map ans;

void init_primes(){
    for(int i = 2; i < maxN; i++) is_prime[i] = true;
    for(int i = 2; i < maxN; i++){
        if(is_prime[i]){
            primes.push_back(i);
            for(int j = 2 * i; j < maxN; j += i)
                is_prime[j] = false;
        }
    }
}

int dfs(int u){
    vis[u] = true;
    if(vis[perm[u]]) return 1;
    return dfs(perm[u]) + 1;
}

void prime_factor(int x){
    for(int i = 0; i < (int) primes.size() && primes[i] <= x; i++){
        int p = primes[i];
        int num_divisions = 0;
        while(x % p == 0){
            x /= p;
            num_divisions++;
        }

        if(num_divisions > 0)
            ans[p] = max(ans[p], num_divisions);
    }
}

int main(){
    init_primes();

    scanf("%d", &N);
    for(int i = 1; i <= N; i++)
        scanf("%d", &perm[i]);

    for(int u = 1; u <= N; u++){
        if(!vis[u]){
            int len = dfs(u);
            prime_factor(len);
        }
    }
}

```

```

ll prod = 1;
for(auto const& [prime, power] : ans)
    for(int i = 0; i < power; i++)
        prod = (prod * prime) % MOD;
printf("%lld\n", prod);
}

```

Bracket Sequences I

```

#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, numerator, denominator;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

ll inverse(ll a){
    return fastpow(a, MOD-2);
}

int main(){
    scanf("%lld", &N);
    if(N&1) printf("0\n");
    else {
        numerator = 1;
        for(int i = 1; i <= N; i++)
            numerator = (numerator * i) % MOD;

        denominator = 1;
        for(int i = 1; i <= N/2; i++)
            denominator = (denominator * i) % MOD;
        denominator = (denominator * denominator) % MOD;
        denominator = (denominator * (N/2+1)) % MOD;

        printf("%lld\n", (numerator*inverse(denominator))%MOD);
    }
}

```

Bracket Sequences II

```

#include

using namespace std;
typedef long long ll;
const int maxN = 1e6+5;
const ll MOD = 1e9+7;

char S[maxN];
int N, M, K;
ll tot, bad, fact[maxN], inv[maxN];

ll inverse(ll x){
    ll res = 1;
    ll b = MOD-2;
    while(b){
        if(b&1) res = (res * x) % MOD;
        x = (x * x) % MOD;
        b >>= 1;
    }
    return res;
}

ll choose(int x, int y){
    return (fact[x] * inv[y] % MOD) * inv[x-y] % MOD;
}

void init(){
    fact[0] = inv[0] = 1;
    for(int i = 1; i <= N; i++){
        fact[i] = (fact[i-1] * i) % MOD;
        inv[i] = (inv[i-1] * inverse(i)) % MOD;
    }
}

int main(){
    scanf("%d %s", &N, S);
    M = (int) strlen(S);
    init();

    int open = 0, closed = 0;
    for(int i = 0; i < M; i++){
        if(S[i] == '(') open++;
        else if(S[i] == ')') closed++;

        if(closed > open){
            printf("0\n");
            return 0;
        }
    }

    if(N&1 || open > N/2){
        printf("0\n");
        return 0;
    }

    tot = choose(N-open-closed, N/2-open);
    bad = choose(N-open-closed, N/2-open-1);
    printf("%lld\n", ((tot-bad)%MOD+MOD)%MOD);
}

```

Counting Necklaces

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

int N, M;
ll ans;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%d %d", &N, &M);
    for(int k = 0; k < N; k++)
        ans = (ans + fastpow(M, __gcd(k, N))) % MOD;
    ans = (ans * fastpow(N, MOD-2)) % MOD;
    printf("%lld\n", ans);
}
```

Counting Grids

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, A, B, C;

ll fastpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b&1)
            res = (res * a) % MOD;
        a = (a * a) % MOD;
        b >>= 1;
    }
    return res;
}

int main(){
    scanf("%lld", &N);
    A = N * N;
    B = (A+3*(N&1))/4;
    C = (A+(N&1))/2;
    printf("%lld\n", ((fastpow(2, A)+2*fastpow(2, B)+fastpow(2, C))*fastpow(4,MOD-2))%MOD);
}
```

Fibonacci Numbers

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, x[2][2], y[2][2];

void mult(ll A[2][2], ll B[2][2]){
    ll C[2][2];
    memset(C, 0, sizeof(C));
    for(int i = 0; i < 2; i++){
        for(int j = 0; j < 2; j++){
            for(int k = 0; k < 2; k++){
                C[i][j] += A[i][k] * B[k][j];
                C[i][j] %= MOD;
            }
        }
    }
    for(int i = 0; i < 2; i++)
        for(int j = 0; j < 2; j++)
            A[i][j] = C[i][j];
}

int main(){
    x[0][1] = x[1][0] = x[1][1] = y[0][0] = y[1][1] = 1;

    scanf("%lld", &N);
    while(N){
        if(N&1)
            mult(y, x);
        mult(x, x);
        N >>= 1;
    }

    printf("%lld\n", y[0][1]);
}
```

Throwing Dice

```
#include

using namespace std;
typedef long long ll;
const ll MOD = 1e9+7;

ll N, x[6][6], y[6][6];

void init(){
    for(int i = 0; i < 6; i++)
        x[0][i] = 1;
    for(int i = 0; i < 5; i++)
        x[i+1][i] = 1;
```

```

    for(int i = 0; i < 6; i++)
        y[i][i] = 1;
}

void mult(ll A[6][6], ll B[6][6]){
    ll C[6][6];
    memset(C, 0, sizeof(C));
    for(int i = 0; i < 6; i++){
        for(int j = 0; j < 6; j++){
            for(int k = 0; k < 6; k++){
                C[i][j] += A[i][k] * B[k][j];
                C[i][j] %= MOD;
            }
        }
    }
    for(int i = 0; i < 6; i++)
        for(int j = 0; j < 6; j++)
            A[i][j] = C[i][j];
}

int main(){
    init();

    scanf("%lld", &N);
    while(N){
        if(N&1)
            mult(y, x);
        mult(x, x);
        N >>= 1;
    }

    printf("%lld\n", y[0][0]);
}

```

Graph Paths I

```

#include

using namespace std;
typedef long long ll;
const int maxN = 100;
const ll MOD = 1e9+7;

int N, M, K, a, b;
ll X[maxN][maxN], Y[maxN][maxN];

void mult(ll A[maxN][maxN], ll B[maxN][maxN]){
    ll C[N][N];
    memset(C, 0, sizeof(C));
    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            for(int k = 0; k < N; k++){
                C[i][j] += A[i][k] * B[k][j];
                C[i][j] %= MOD;
            }
        }
    }
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            A[i][j] = C[i][j];
}

```

```

int main(){
    scanf("%d %d %d", &N, &M, &K);
    for(int i = 0; i < M; i++){
        scanf("%d %d", &a, &b);
        X[a-1][b-1]++;
    }
    for(int i = 0; i < N; i++)
        Y[i][i] = 1;

    while(K){
        if(K&1)
            mult(Y, X);
        mult(X, X);
        K >>= 1;
    }

    printf("%lld\n", Y[0][N-1]);
}

```

Graph Paths II

```

#include

using namespace std;
typedef unsigned long long ull;
const int maxN = 100;
const ull INF = 1e19;

int N, M, K, a, b;
ull c, X[maxN][maxN], Y[maxN][maxN];

void combine(ull A[maxN][maxN], ull B[maxN][maxN]){
    ull C[maxN][maxN];
    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            C[i][j] = INF;

    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            for(int k = 0; k < N; k++)
                if(A[i][k] != INF && B[k][j] != INF)
                    C[i][j] = min(C[i][j], A[i][k] + B[k][j]);

    for(int i = 0; i < N; i++)
        for(int j = 0; j < N; j++)
            A[i][j] = C[i][j];
}

int main(){
    scanf("%d %d %d", &N, &M, &K);

    for(int i = 0; i < N; i++){
        for(int j = 0; j < N; j++){
            X[i][j] = Y[i][j] = INF;
            Y[i][i] = 0;
        }
    }

    for(int i = 0; i < M; i++){
        scanf("%d %d %llu", &a, &b, &c);
        X[a-1][b-1] = min(X[a-1][b-1], c);
    }
}

```

```

    }

    while(K){
        if(K&1)
            combine(Y, X);
        combine(X, X);
        K >>= 1;
    }

    if(Y[0][N-1] == INF)    printf("-1\n");
    else                    printf("%llu\n", Y[0][N-1]);
}

```

Sum of Four Squares

```

#include

using namespace std;
typedef pair pii;
typedef array solution;
const int maxN = 1e7+1;

set squares;
map small_solutions;
bool prime[maxN];

void init_squares(){
    for(int i = 0; i * i <= maxN; i++)
        squares.insert(i * i);
}

void init_primes(){
    for(int i = 2; i < maxN; i++)    prime[i] = true;
    for(int i = 2; i * i < maxN; i++)
        if(prime[i])
            for(int j = 2*i; j < maxN; j += i)
                prime[j] = false;
}

void precompute_small(){
    for(int a = 0; a <= 100; a++)
        for(int b = 0; b <= 100; b++)
            for(int c = 0; c <= 20; c++)
                for(int d = 0; d <= 5; d++)
                    small_solutions[a*a + b*b + c*c + d*d] = {a, b, c, d};
}

// Returns a and b such that
// a and b have opposite parity
// and p := n - a^2 - b^2 is prime
pii find_ab(int n){
    int sqrtn = (int) sqrt(n);

    while(true){
        int a = (rand() % sqrtn) + 1;
        int b = (rand() % sqrtn) + 1;

        if((a+b) % 2 == 0)    a++;

        int p = n - a*a - b*b;
        if(p >= 2 && prime[p])

```

```

        return {a, b};
    }
}

// If prime p is congruent to 1 (mod 4),
// then there exists integers (c, d)
// such that  $c^2 + d^2 = p$ .
// Naively find such c and d.
pii find_cd(int p){
    for(int c = 0; c*c <= p; c++){
        int d_squared = p - c*c;
        if(squares.find(d_squared) != squares.end())
            return {c, (int) sqrt(d_squared)};
    }
    return {-1, -1};
}

solution solve(int n){
    if(small_solutions.find(n) != small_solutions.end()){
        return small_solutions[n];
    } else if(n == 0){
        return {0, 0, 0, 0};
    } else if(n % 4 == 0){
        solution sol = solve(n / 4);
        for(int i = 0; i < 4; i++) sol[i] *= 2;
        return sol;
    } else if(n % 4 == 1 || n % 4 == 3){
        solution sol = solve(2 * n);
        sort(sol.begin(), sol.end(), [](int a, int b){
            return (a % 2 > b % 2);
        });
        return {
            (sol[0] + sol[1]) / 2,
            (sol[0] - sol[1]) / 2,
            (sol[2] + sol[3]) / 2,
            (sol[2] - sol[3]) / 2,
        };
    } else {
        pii ab = find_ab(n);
        int a = ab.first, b = ab.second;
        int p = n - a*a - b*b;
        pii cd = find_cd(p);
        int c = cd.first, d = cd.second;
        return {a, b, c, d};
    }
}

int main(){
    init_squares();
    init_primes();
    precompute_small();

    int T;
    scanf("%d", &T);
    for(int t = 0, n; t < T; t++){
        scanf("%d", &n);
        solution sol = solve(n);
        for(int i = 0; i < 4; i++) sol[i] = abs(sol[i]);
        printf("%d %d %d %d\n", sol[0], sol[1], sol[2], sol[3]);
    }
}

```

Triangle Number Sums

```
#include

using namespace std;
typedef long long ll;
const ll maxN = (ll) 1e12;

unordered_set S;
vector nums;

bool is_two_sum(ll n){
    int l = 0, r = (int) nums.size() - 1;
    while(l < (int) nums.size() && r >= 0){
        ll cur = nums[l] + nums[r];
        if(cur == n) return true;
        else if(cur < n) l++;
        else r--;
    }
    return false;
}

int answer(ll n){
    if(S.find(n) != S.end()) return 1;
    else if(is_two_sum(n)) return 2;
    else return 3;
}

int main(){
    S.reserve((int) 1e6);
    for(ll i = 1, num = 1; num <= maxN; i++, num += i){
        nums.push_back(num);
        S.insert(num);
    }

    int T;
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        ll n;
        scanf("%lld", &n);
        printf("%d\n", answer(n));
    }
}
```

Dice Probability

```
#include

using namespace std;
const int maxN = 100;

int N, a, b;
double sum, dp[maxN+1][6*maxN+1];

int main(){
    scanf("%d %d %d", &N, &a, &b);
    dp[0][0] = 1;
    for(int i = 1; i <= N; i++)
```

```

    for(int j = 1; j <= 6*maxN; j++)
        for(int k = 1; k <= 6; k++)
            if(j-k >= 0)
                dp[i][j] += dp[i-1][j-k]/6;
    for(int i = a; i <= b; i++)
        sum += dp[N][i];
    printf("%.6f\n", sum);
}

```

Moving Robots

```

#include

using namespace std;
const int maxK = 101;
const int N = 8, SZ = N*N;

int K;
double expected, ans[SZ], dp[maxK][SZ];

int main(){
    scanf("%d", &K);
    for(int i = 0; i < SZ; i++)
        ans[i] = 1.0;

    for(int start = 0; start < SZ; start++){
        memset(dp, 0, sizeof(dp));
        dp[0][start] = 1.0;

        for(int k = 0; k < K; k++){
            for(int u = 0; u < SZ; u++){
                vector V;
                if(N <= u)          V.push_back(u-N);
                if(u < N*(N-1))    V.push_back(u+N);
                if(u % N != 0)     V.push_back(u-1);
                if(u % N != N-1)  V.push_back(u+1);

                for(int v : V)
                    dp[k+1][v] += (dp[k][u] / V.size());
            }
        }

        for(int u = 0; u < SZ; u++)
            ans[u] *= (1 - dp[K][u]);
    }

    for(int i = 0; i < SZ; i++)
        expected += ans[i];
    printf("%.6f\n", expected);
}

```

Candy Lottery

```
#include

using namespace std;

int N, K;
double ans, a, b;

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 1; i <= K; i++){
        a = b = 1.0;
        for(int j = 1; j <= N; j++){
            a *= (double) i / K;
            b *= (double) (i-1) / K;
        }
        ans += (a-b) * i;
    }
    printf("%.6f\n", ans);
}
```

Inversion Probability

```
#include

using namespace std;
typedef long long ll;
const int maxN = 101;

int N, r[maxN];
long double ans;

int f(int x){
    return x * (x-1) / 2;
}

int main(){
    scanf("%d", &N);
    for(int i = 0; i < N; i++){
        scanf("%d", &r[i]);
        for(int j = 0; j < i; j++){
            int tot = (r[j] <= r[i]) ? f(r[j]) : (f(r[i]) + (r[j]-r[i]) * r[i]);
            ans += (long double) tot / (r[i] * r[j]);
        }
    }
    printf("%.6Lf\n", ans);
}
```

Stick Game

```
#include
```

```
using namespace std;
const int maxN = 1e6;
const int maxK = 100;

int N, K, p[maxK];
bool dp[maxN+1];

int main(){
    scanf("%d %d", &N, &K);
    for(int i = 0; i < K; i++)
        scanf("%d", &p[i]);

    for(int i = 1; i <= N; i++){
        for(int j = 0; j < K; j++){
            if(i-p[j] >= 0 && !dp[i-p[j]])
                dp[i] = true;
            printf("%c", ("LW")[dp[i]]);
        }
    }
}
```

Nim Game I

```
#include

using namespace std;

int T, N, x, xum;

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        scanf("%d", &N);
        xum = 0;
        for(int i = 0; i < N; i++){
            scanf("%d", &x);
            xum ^= x;
        }
        printf("%s\n", (xum ? "first" : "second"));
    }
}
```

Nim Game II

```
#include

using namespace std;

int T, N, x, xum;

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
```

```

scanf("%d", &N);
xum = 0;
for(int i = 0; i < N; i++){
    scanf("%d", &x);
    xum ^= (x%4);
}
printf(xum ? "first\n" : "second\n");
}

```

Stair Game

```

#include

using namespace std;

int T, N, p, xum;

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        scanf("%d", &N);
        xum = 0;
        for(int i = 0; i < N; i++){
            scanf("%d", &p);
            if(i%2)
                xum ^= p;
        }
        printf(xum ? "first\n" : "second\n");
    }
}

```

Grundy's Game

```

#include

using namespace std;
const int maxN = 1e6+1;

/**
 * A036685
 * Retrived from https://oeis.org/A036685
 */

int T, lose[42] = { 0, 1, 2, 4, 7, 10, 20, 23, 26, 50, 53, 270, 273, 276, 282, 285, 288, 316, 334, 337, 340, 346, 359, 362 };
bool b[maxN];

void init(){
    for(int x : lose)
        b[x] = true;
}

```

```
int main(){
    init();
    scanf("%d", &T);
    for(int t = 0, N; t < T; t++){
        scanf("%d", &N);
        printf("%s\n", b[N] ? "second" : "first");
    }
}
```

Another Game

```
#include

using namespace std;

int T, N, x;
bool even;

int main(){
    scanf("%d", &T);
    for(int t = 0; t < T; t++){
        scanf("%d", &N);
        even = true;
        for(int i = 0; i < N; i++){
            scanf("%d", &x);
            even &= !(x&1);
        }
        printf("%s\n", even ? "second" : "first");
    }
}
```